

JSON Structure And Analytics Database Schema

This document describes the raw MKW Table Bot JSON files currently in `backend/JSON` and proposes a normalized database schema for analytics across multiple CTC seasons and divisions.

Raw files are organized by league, season, and division:

```
backend/JSON/{league}/{season}/{division}/
```

Examples:

- `backend/JSON/ctc/s1/d1_2/`
- `backend/JSON/ctc/s1/d3/`
- `backend/JSON/ctc/s1/d4/`

Future uploads should use the same structure, with one division number per folder unless the historical season truly combined divisions.

Analysis Summary

The CTC match files are JSON objects even when the file extension is `.txt`. Most match files also have a `.json` copy. Across the current repo, the match-shaped files have a consistent structure:

- top-level match metadata
- an ordered track list
- one `teams` object keyed by team tag/name
- each team has a `players` object keyed by friend code
- each player has per-race scores and positions

One notable outlier is `backend/JSON/rt_gsc_s13/rt_gsc_s13.json`, which is a top-level list containing many match-like objects rather than one match object. The importer should support both single-match JSON objects and batch JSON arrays.

Observed match-shaped file facts:

- Top-level keys: `title_str`, `format`, `racess_played`, `rxs`, `tracks`, `teams`
- Observed `format`: `5v5`
- Observed race counts: `7`, `10`, `11`, `12`, `13`, `17`
- Observed teams per match: `1`, `2`, `3`
- Observed players per team object: `1`, `4`, `5`, `6`, `7`
- Some team records have penalties.
- A smaller number of player records have penalties.
- Some players have `subbed_out`: `true`.
- Some `race_positions` arrays contain `null`.

Season and division are not present inside the JSON. They need to come from folder naming, filename metadata, an upload form, or a manual import manifest.

CTC matches should be modeled as two-team matches. If an imported JSON object has a third team, treat that as a data-quality issue rather than a true three-team match. The likely explanation is that a sub was parsed as a separate team by mistake and should be manually assigned to one of the two actual teams during import review.

Raw JSON Shape

Representative structure:

```
{
  "title_str": "#title 12 races\n",
  "format": "5v5",
  "races_played": 12,
  "rxx": ["r10286270", "r10286376"],
  "tracks": ["Banished Keep", "Pipe Underworld"],
  "teams": {
    "TMNG": {
      "table_tag_str": "TMNG #FF0404",
      "table_penalty_str": "",
      "total_score": 334,
      "penalties": 0,
      "hex_color": "#FF0404",
      "players": {
        "5245-8888-2222": {
          "table_str": "Mossan [jp] 35|41|40",
          "mii_name": "TMNG shun",
          "lounge_name": "Mossan",
          "table_name": "Mossan",
          "tag": "TMNG",
          "total_score": 116,
          "had_penalties": false,
          "penalties": 0,
          "subbed_out": false,
          "race_scores": [10, 4],
          "race_positions": [3, 6],
          "gp_scores": [[10, 4]],
          "flag": "jp"
        }
      }
    }
  }
}
```

Field Meaning

Match-Level Fields

JSON field	Meaning	Notes
title_str	Table title string	Often contains race count text.

JSON field	Meaning	Notes
<code>format</code>	Match format	Currently observed as <code>5v5</code> .
<code>races_played</code>	Number of races in this table	Should match <code>tracks.length</code> in normal cases.
<code>rx</code>	Table Bot race/table references	Array length varies from 1 to 4. Store as child rows.
<code>tracks</code>	Ordered list of tracks	The array index maps to race number.
<code>teams</code>	Team result object	Keys are raw team tags/names.

Team-Level Fields

JSON field	Meaning	Notes
object key	Raw team key	Example <code>TMNG</code> ; may be clan tag or display tag.
<code>table_tag_str</code>	Tag plus color string	Example <code>TMNG #FF0404</code> .
<code>table_penalty_str</code>	Human-readable penalty string	Empty string if none.
<code>total_score</code>	Team total score shown by table bot	Store raw. Confirm later whether already net of penalties.
<code>penalties</code>	Team penalty points	Usually <code>0</code> , positive number when penalty exists.
<code>hex_color</code>	Team display color	Useful for frontend but not identity.
<code>players</code>	Player result object	Keys are friend codes.

Player-Level Fields

JSON field	Meaning	Notes
object key	Friend code	Best available stable player identifier from JSON.
<code>table_str</code>	Rendered table row text	Useful for audit/debugging.
<code>mii_name</code>	Raw Mii name	Can include symbols and team tag styling.
<code>lounge_name</code>	Lounge name	Sometimes empty.
<code>table_name</code>	Displayed table name	Fallback when lounge name is empty.
<code>tag</code>	Team tag on player row	Usually matches team key.
<code>total_score</code>	Player total score	Store raw.
<code>had_penalties</code>	Whether player had penalties	Rare, but present.
<code>penalties</code>	Player penalty points	Usually <code>0</code> .
<code>subbed_out</code>	Whether player subbed out	Store for roster and race availability logic.
<code>race_scores</code>	Ordered per-race scores	Index maps to <code>tracks</code> index.
<code>race_positions</code>	Ordered per-race placements	May contain <code>null</code> .

JSON field	Meaning	Notes
<code>gp_scores</code>	Scores grouped by GP	Can be stored as JSON or normalized later.
<code>flag</code>	Country/region flag code	Optional profile attribute.

Recommended Database Model

The main idea is to keep raw imported facts separate from normalized identities:

- `teams` and `players` represent canonical identities.
- `team_season_entries` and `player_season_entries` represent who they were in a specific season/division.
- `matches`, `match_teams`, and `match_players` preserve exactly what the table showed.
- `races` and `race_player_results` power analytics.
- `penalties` stores penalty facts without pretending all penalties are race-level when the JSON does not say that.

Core Tables

`seasons`

Stores CTC season metadata.

Column	Type	Notes
<code>season_id</code>	integer primary key	Internal ID.
<code>season_number</code>	integer	Example 1, 2, 3.
<code>name</code>	text	Example <code>Custom Track Cup Season 3</code> .
<code>status</code>	text	Example <code>complete</code> , <code>ongoing</code> , <code>planned</code> .
<code>starts_on</code>	date nullable	Optional.
<code>ends_on</code>	date nullable	Optional.

`divisions`

Stores divisions within a season.

Column	Type	Notes
<code>division_id</code>	integer primary key	Internal ID.
<code>season_id</code>	foreign key to <code>seasons</code>	Required.
<code>division_code</code>	text	Example 1_2, 3, 4.
<code>division_name</code>	text	Example <code>Division 1-2</code> .

Recommended unique key: (`season_id`, `division_code`).

source_files

Tracks every imported raw file.

Column	Type	Notes
source_file_id	integer primary key	Internal ID.
season_id	foreign key to seasons	Placeholder/manual value at import.
division_id	foreign key to divisions	Placeholder/manual value at import.
source_path	text	Repo path or upload path.
source_filename	text	Original filename.
file_sha256	text	Deduplication and audit.
json_shape	text	Example single_match or match_array .
imported_at	timestamp	When processed.

Team Tables

teams

Canonical team/clan identity.

Column	Type	Notes
team_id	integer primary key	Internal ID.
canonical_name	text	Human-friendly name if known.
canonical_tag	text	Stable clan tag if known.
created_at	timestamp	Audit.

This table should not be overtrusted early. Clan tags and names can be messy, so start with raw imported tags and allow manual merges later.

team_season_entries

Team identity within one season/division.

Column	Type	Notes
team_season_entry_id	integer primary key	Internal ID.
team_id	foreign key to teams	Canonical team.
season_id	foreign key to seasons	Required.
division_id	foreign key to divisions	Required.
display_name	text	Name to show for this season/division.

Column	Type	Notes
<code>clan_tag</code>	text	Raw or normalized tag.
<code>hex_color</code>	text nullable	Most recently seen table color.

Recommended unique key: (`season_id`, `division_id`, `clan_tag`), with manual override support if needed.

Player Tables

`players`

Canonical player/person identity.

Column	Type	Notes
<code>player_id</code>	integer primary key	Internal ID.
<code>canonical_lounge_name</code>	text nullable	Best display name.
<code>primary_friend_code</code>	text nullable	Friend codes can change, so keep aliases too.
<code>created_at</code>	timestamp	Audit.

`player_friend_codes`

Stores all friend codes seen for a player.

Column	Type	Notes
<code>player_friend_code_id</code>	integer primary key	Internal ID.
<code>player_id</code>	foreign key to <code>players</code>	Required.
<code>friend_code</code>	text	Example <code>5245-8888-2222</code> .
<code>first_seen_match_id</code>	foreign key nullable	Optional audit.
<code>last_seen_match_id</code>	foreign key nullable	Optional audit.

Recommended unique key: `friend_code`, unless you later discover recycled/shared codes.

`player_aliases`

Stores lounge names, Mii names, and table names seen for a player.

Column	Type	Notes
<code>player_alias_id</code>	integer primary key	Internal ID.
<code>player_id</code>	foreign key to <code>players</code>	Required.
<code>alias_type</code>	text	<code>lounge_name</code> , <code>mii_name</code> , <code>table_name</code> , <code>manual</code> .

Column	Type	Notes
<code>alias_value</code>	text	Raw value.
<code>first_seen_match_id</code>	foreign key nullable	Optional audit.
<code>last_seen_match_id</code>	foreign key nullable	Optional audit.

player_season_entries

Connects players to teams for a specific season/division.

Column	Type	Notes
<code>player_season_entry_id</code>	integer primary key	Internal ID.
<code>player_id</code>	foreign key to <code>players</code>	Required.
<code>team_season_entry_id</code>	foreign key to <code>team_season_entries</code>	Required.
<code>season_id</code>	foreign key to <code>seasons</code>	Denormalized for easy filtering.
<code>division_id</code>	foreign key to <code>divisions</code>	Denormalized for easy filtering.
<code>primary_lounge_name</code>	text nullable	Best name in this season/division.
<code>primary_mii_name</code>	text nullable	Optional.
<code>flag</code>	text nullable	Latest seen flag code.
<code>first_seen_match_id</code>	foreign key nullable	Optional audit.
<code>last_seen_match_id</code>	foreign key nullable	Optional audit.

This table handles the case where a player changes teams between seasons or divisions.

Match Tables

matches

One imported match/table.

Column	Type	Notes
<code>match_id</code>	integer primary key	Internal ID.
<code>season_id</code>	foreign key to <code>seasons</code>	Manual/import metadata.
<code>division_id</code>	foreign key to <code>divisions</code>	Manual/import metadata.
<code>source_file_id</code>	foreign key to <code>source_files</code>	Audit.
<code>week_number</code>	integer nullable	Can be parsed from filenames like <code>W1</code> , or entered manually.
<code>match_label</code>	text	Filename or friendly label.

Column	Type	Notes
<code>title_str</code>	text nullable	Raw JSON title.
<code>format</code>	text	Example <code>5v5</code> .
<code>races_played</code>	integer	From JSON.
<code>raw_json</code>	json nullable	Optional full raw snapshot.
<code>created_at</code>	timestamp	Audit.

Validation rule: a normal imported CTC match should resolve to exactly two `match_teams` rows. If the raw JSON contains one or three team objects, mark the match as needing review before analytics treat it as final.

`match_table_refs`

Stores `rx` values.

Column	Type	Notes
<code>match_table_ref_id</code>	integer primary key	Internal ID.
<code>match_id</code>	foreign key to <code>matches</code>	Required.
<code>ref_value</code>	text	Example <code>r10286270</code> .
<code>ref_order</code>	integer	Original order in <code>rx</code> .

`match_teams`

One row per team in a match.

Column	Type	Notes
<code>match_team_id</code>	integer primary key	Internal ID.
<code>match_id</code>	foreign key to <code>matches</code>	Required.
<code>team_season_entry_id</code>	foreign key to <code>team_season_entries</code>	Required after identity resolution.
<code>raw_team_key</code>	text	Key from the <code>teams</code> object.
<code>table_tag_str</code>	text nullable	Raw JSON.
<code>hex_color</code>	text nullable	Raw JSON.
<code>raw_total_score</code>	integer	From JSON.
<code>team_penalty_points</code>	integer default 0	From JSON <code>penalties</code> .
<code>table_penalty_str</code>	text nullable	Raw penalty text.
<code>final_score</code>	integer nullable	Optional computed/confirmed score.

`match_players`

One row per player as listed on a match table.

Column	Type	Notes
<code>match_player_id</code>	integer primary key	Internal ID.
<code>match_team_id</code>	foreign key to <code>match_teams</code>	Required.
<code>player_id</code>	foreign key to <code>players</code>	Required after identity resolution.
<code>player_season_entry_id</code>	foreign key nullable	Useful when roster is known.
<code>friend_code_raw</code>	text	Object key from JSON.
<code>lounge_name_raw</code>	text nullable	Raw JSON.
<code>mii_name_raw</code>	text nullable	Raw JSON.
<code>table_name_raw</code>	text nullable	Raw JSON.
<code>tag_raw</code>	text nullable	Raw JSON.
<code>flag</code>	text nullable	Raw JSON.
<code>table_str</code>	text nullable	Raw rendered table row.
<code>raw_total_score</code>	integer	From JSON.
<code>player_penalty_points</code>	integer default 0	From JSON.
<code>had_penalties</code>	boolean	From JSON.
<code>subbed_out</code>	boolean	From JSON.
<code>gp_scores_json</code>	json nullable	Preserve GP grouping.

Race And Result Tables

`tracks`

Canonical track identity.

Column	Type	Notes
<code>track_id</code>	integer primary key	Internal ID.
<code>canonical_name</code>	text	Normalized track name.
<code>created_at</code>	timestamp	Audit.

Recommended unique key: `canonical_name`.

`track_aliases`

Stores raw spellings or future renamed tracks.

Column	Type	Notes
<code>track_alias_id</code>	integer primary key	Internal ID.
<code>track_id</code>	foreign key to <code>tracks</code>	Required.
<code>alias_value</code>	text	Raw imported value.

`racess`

One row per race in a match.

Column	Type	Notes
<code>race_id</code>	integer primary key	Internal ID.
<code>match_id</code>	foreign key to <code>matches</code>	Required.
<code>race_number</code>	integer	1-based index from <code>tracks</code> .
<code>track_id</code>	foreign key to <code>tracks</code>	Required after track resolution.
<code>track_name_raw</code>	text	Raw value from JSON <code>tracks</code> .
<code>has_penalty</code>	boolean default <code>false</code>	Derived from penalties linked to this race, if known.

Recommended unique key: `(match_id, race_number)`.

Important caveat: the current JSON usually does not identify which race a penalty belongs to. Most penalties are team-level or player-level match penalties. For that reason, detailed penalty data should live in a separate `penalties` table where `race_id` can be nullable.

`race_player_results`

One row per player's result in one race.

Column	Type	Notes
<code>race_player_result_id</code>	integer primary key	Internal ID.
<code>race_id</code>	foreign key to <code>racess</code>	Required.
<code>match_player_id</code>	foreign key to <code>match_players</code>	Required.
<code>player_id</code>	foreign key to <code>players</code>	Denormalized for fast queries.
<code>match_team_id</code>	foreign key to <code>match_teams</code>	Denormalized for fast queries.
<code>team_season_entry_id</code>	foreign key to <code>team_season_entries</code>	Denormalized for fast queries.
<code>score</code>	integer nullable	From <code>race_scores[race_number - 1]</code> .
<code>position</code>	integer nullable	From <code>race_positions[race_number - 1]</code> ; can be null.

Column	Type	Notes
<code>role</code>	text	<code>runner</code> , <code>bagger</code> , or <code>unknown</code> .
<code>role_source</code>	text	<code>manual</code> , <code>inferred</code> , or <code>unknown</code> .
<code>is_subbed_out_result</code>	boolean	Useful when score/position is missing or null.

For the bagging/running flag, the JSON does not explicitly say "bagger" or "runner". The current CSV extractor infers bagging when a race score is exactly `1`, but that should not be treated as permanent truth. Store an inferred role, then allow manual correction.

Suggested initial inference:

- `score = 1`: `role = bagger`, `role_source = inferred`
- otherwise: `role = runner`, `role_source = inferred`
- if score or position is missing/null: `role = unknown`, `role_source = unknown`

If CTC has a better official rule, encode that rule in the importer and keep `role_source` so future audits are possible.

Penalty Tables

`penalties`

Stores team-level, player-level, and optionally race-level penalties.

Column	Type	Notes
<code>penalty_id</code>	integer primary key	Internal ID.
<code>match_id</code>	foreign key to <code>matches</code>	Required.
<code>race_id</code>	foreign key nullable	Null when JSON does not identify a race.
<code>match_team_id</code>	foreign key nullable	Set for team penalties.
<code>match_player_id</code>	foreign key nullable	Set for player penalties.
<code>penalty_scope</code>	text	<code>team</code> , <code>player</code> , <code>race</code> , <code>unknown</code> .
<code>penalty_points</code>	integer	Positive points deducted, matching JSON style.
<code>raw_penalty_text</code>	text nullable	Example <code>Penalty -5</code> .
<code>source_field</code>	text	Example <code>team.penalties</code> , <code>player.penalties</code> .

This table keeps the `races.has_penalty` flag simple while preserving the true source detail.

Import Mapping

For each match object:

1. Create or find `season` and `division` from provided import metadata.

2. Create `source_files` row using file path and hash.
3. Create `matches` row.
4. Insert each `rx` value into `match_table_refs`.
5. Insert or resolve each raw team into `teams` and `team_season_entries`.
6. Insert `match_teams`, but require the match to resolve to exactly two real teams.
7. Insert team penalties into `penalties`.
8. Insert or resolve each player by friend code into `players`.
9. Store lounge, Mii, and table names in `player_aliases`.
10. Connect player to team/season/division in `player_season_entries`.
11. Insert `match_players`.
12. Insert player penalties into `penalties`.
13. Insert tracks into `tracks` and `track_aliases`.
14. Insert one `races` row per track index.
15. Expand `race_scores` and `race_positions` into `race_player_results`.

Suggested Minimum Viable Schema

If you want to move quickly, start with these tables:

- `seasons`
- `divisions`
- `source_files`
- `teams`
- `team_season_entries`
- `players`
- `player_aliases`
- `player_season_entries`
- `matches`
- `match_teams`
- `match_players`
- `tracks`
- `races`
- `race_player_results`
- `penalties`

This is enough to power the existing website analytics and unlock season/division filtering, match-level auditability, player/team history, and race-level analytics.

Design Notes

- Do not use team tag alone as a permanent team ID.
- Model CTC matches as two-team matches; third raw team objects should enter a review queue and usually represent a sub attached to the wrong team.
- Do not use lounge name alone as a permanent player ID.
- Friend code is the best player identity signal in the current JSON, but keep alias tables because names and codes can change.
- Keep raw names and raw table strings so imports are auditable.

- Keep `raw_json` or at least `source_files.file_sha256` so every derived row can be traced back to a source.
- Keep season and division as explicit database fields even if they are manually supplied at first.
- Store penalties separately from races because the JSON does not generally locate penalties by race.
- Store bagger/runner role as a value with a source, not as a destructive rename like `player (bag)`.